

# BÁO CÁO THỰC HÀNH

TRIỂN KHAI VÀ QUẢN TRỊ  
HỆ THỐNG PHẦN MỀM

## ĐỀ 6: HỆ THỐNG ĐẶT PHÒNG KHÁCH SẠN LOTUS STAY

| Thông tin            | Nội dung                 |
|----------------------|--------------------------|
| Sinh viên            | [Điền họ và tên]         |
| Mã số sinh viên      | [Điền MSSV]              |
| Lớp                  | [Điền lớp]               |
| Giảng viên hướng dẫn | [Điền họ tên giảng viên] |
| Repository GitHub    | [Điền URL repository]    |

Thái Nguyên, tháng 10 năm 2026

## MỤC LỤC VÀ PHẠM VI

1. Mục tiêu và đối chiếu tiêu chí
2. Kiến trúc và cách thức hoạt động
3. Thiết kế dữ liệu và phân quyền
4. Website và nghiệp vụ đặt phòng
5. Docker Compose và triển khai
6. Nginx reverse proxy
7. Giám sát Prometheus – Grafana
8. Log tập trung Loki – Promtail
9. Hardening hệ thống
10. Kiểm thử và đánh giá kết quả
11. GitHub, vận hành và demo
12. Kết luận, tài liệu tham khảo và minh chứng

### Phạm vi sản phẩm

Hệ thống phục vụ bài thực hành triển khai một website đặt phòng khách sạn duy nhất. Khách hàng xem danh mục phòng, tra cứu ngày còn trống, tạo tài khoản, đặt chỗ và hủy đơn của mình. Quản trị viên quản lý phòng, người dùng và trạng thái đặt chỗ. Phạm vi không bao gồm thanh toán trực tuyến, gửi email xác nhận hay tích hợp hệ thống khách sạn bên ngoài.

### Trạng thái bàn giao

Mã nguồn, cấu hình và hướng dẫn đã được chuẩn bị. Bộ 15 kiểm thử nghiệp vụ đã chạy với SQLite trong môi trường phát triển. Kết quả Docker Compose, PostgreSQL thật, dashboard và log cần được xác nhận trên máy triển khai; ảnh minh chứng thực tế được bổ sung theo danh sách ở cuối báo cáo. Các sơ đồ trong báo cáo mô tả thiết kế, không thay thế ảnh chạy hệ thống.

## 1. MỤC TIÊU VÀ ĐỐI CHIẾU TIÊU CHÍ

Mục tiêu của đề tài là triển khai được một hệ thống web có cơ sở dữ liệu, công cụ quản trị dữ liệu, reverse proxy, metrics và log tập trung trong một cấu hình Docker Compose thống nhất. Sinh viên phải nắm được đường đi của yêu cầu, vai trò của từng dịch vụ và cách sử dụng minh chứng để chứng minh hệ thống hoạt động.

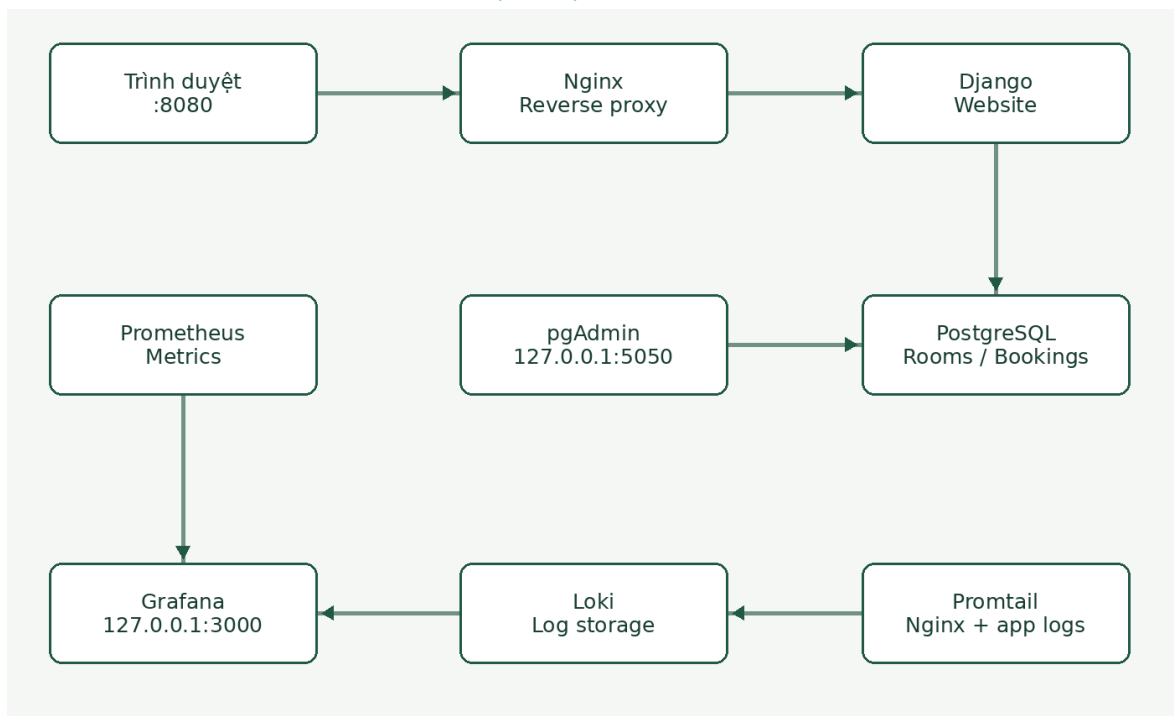
| Tiêu chí             | Thành phần bàn giao              | Minh chứng cần có            |
|----------------------|----------------------------------|------------------------------|
| <b>GitHub</b>        | Source, README, lịch sử 3 commit | URL repo và lịch sử commit   |
| <b>Ứng dụng + DB</b> | Django, PostgreSQL, pgAdmin      | Đơn đặt phòng và bản ghi DB  |
| <b>Reverse proxy</b> | Nginx, security headers          | Truy cập :8080, HTTP headers |
| <b>Giám sát</b>      | Prometheus, Grafana, 3 exporter  | Targets UP và biểu đồ        |
| <b>Log</b>           | Loki, Promtail, 5 query mẫu      | Ít nhất 3 kết quả LogQL      |
| <b>Hardening</b>     | UID, mạng, DB roles, headers     | Lệnh kiểm tra và cấu hình    |
| <b>Trình bày</b>     | README, checklist, báo cáo       | Demo đầy đủ các luồng        |

### Tiêu chuẩn hoàn thành

Website phải truy cập qua Nginx, ghi dữ liệu vào PostgreSQL và đọc được bản ghi bằng pgAdmin. Prometheus phải thu được số liệu của container, web server và database; việc exporter chạy chưa đủ, cần kiểm tra thêm pg\_up và nginx\_up. Grafana phải có dữ liệu trong khoảng thời gian đang chọn. Loki phải trả về log phát sinh thực tế, không chỉ hiển thị datasource đã kết nối.

Tính tái lập được thể hiện bằng việc một máy Ubuntu khác có Docker Compose và các file dự án có thể khởi động hệ thống theo README. Các mật khẩu được sinh tại máy triển khai thay vì lưu sẵn trong mã nguồn. Dữ liệu phải còn sau thao tác dừng rồi mở lại container.

## 2. KIẾN TRÚC VÀ CÁCH THỨC HOẠT ĐỘNG



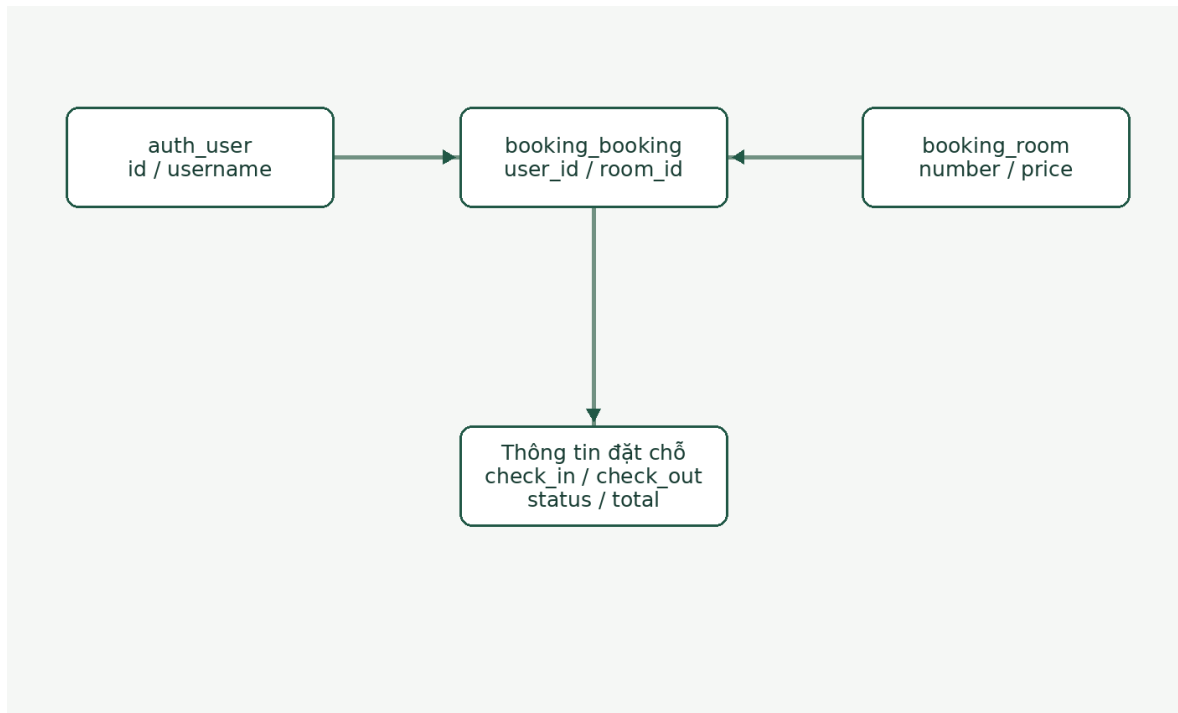
Hình 1. Kiến trúc logic của hệ thống Lotus Stay.

Trình duyệt gửi HTTP tới cổng 8080 của Nginx. Nginx phục vụ file tĩnh và chuyển yêu cầu động sang Gunicorn/Django ở cổng 8000 trong mạng Docker. Django xử lý session, kiểm tra CSRF, xác thực dữ liệu rồi truy cập PostgreSQL. PostgreSQL không có cổng xuất trực tiếp ra host.

Nhánh quan sát được tách khỏi luồng nghiệp vụ. Prometheus lấy metrics từ cAdvisor, Nginx exporter, PostgreSQL exporter và endpoint metrics nội bộ của ứng dụng. Grafana truy vấn Prometheus để vẽ biểu đồ. Promtail đọc log ứng dụng và Nginx từ các volume dùng chung, đẩy vào Loki; Grafana Explore truy vấn Loki bằng LogQL.

Bốn mạng Docker gồm edge, data, observe và admin. Mạng data và observe được đặt internal. Dịch vụ chỉ tham gia những mạng cần thiết: PostgreSQL thuộc data; Nginx thuộc edge/observe; web thuộc edge/data/observe. pgAdmin tham gia data để kết nối DB và admin để phục vụ giao diện quản trị.

### 3. THIẾT KẾ DỮ LIỆU VÀ PHÂN QUYỀN



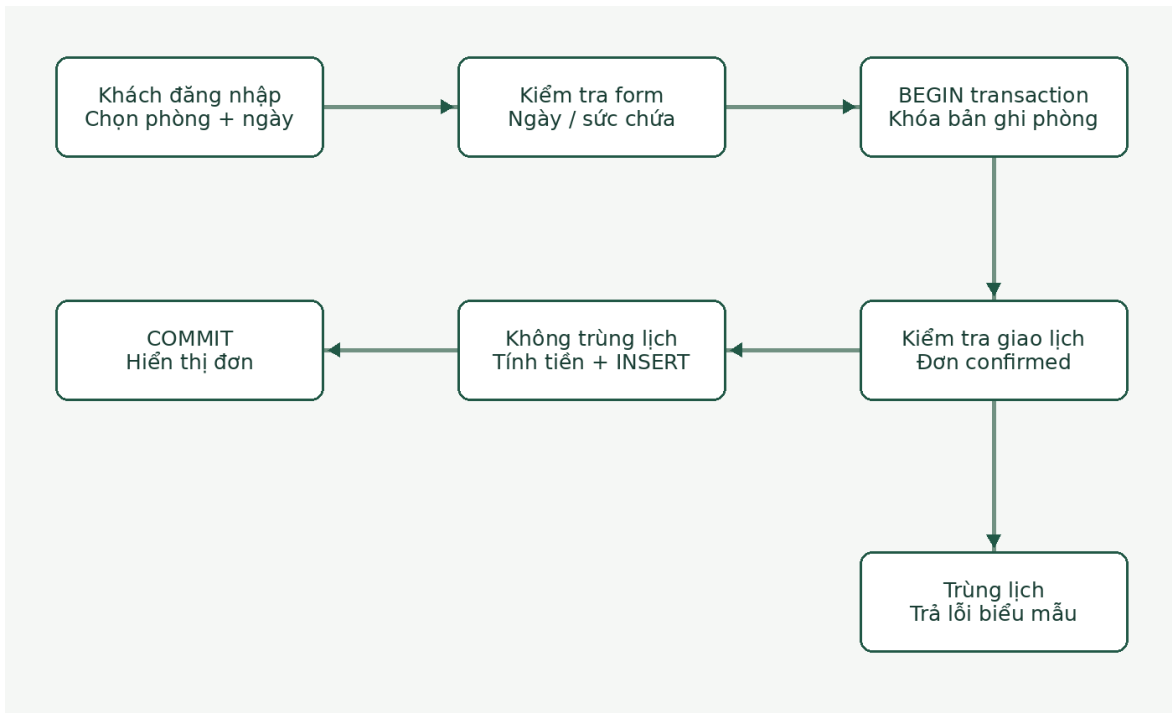
Hình 2. Quan hệ dữ liệu: người dùng và phòng có nhiều đặt chỗ.

| Bảng                   | Dữ liệu chính                                            |
|------------------------|----------------------------------------------------------|
| <b>auth_user</b>       | Tài khoản, mật khẩu băm, quyền staff/superuser           |
| <b>booking_room</b>    | Số phòng duy nhất, hạng, sức chứa, giá, trạng thái       |
| <b>booking_booking</b> | Người đặt, phòng, khách, ngày nhận/trả, tiền, trạng thái |

Khóa ngoại từ Booking tới User và Room dùng PROTECT, ngăn xóa dữ liệu được tham chiếu bởi lịch sử đặt chỗ. Ràng buộc cơ sở dữ liệu yêu cầu giá và sức chứa dương, số khách dương và ngày trả sau ngày nhận. Giá được chụp thành tổng tiền tại thời điểm đặt; thay đổi giá phòng về sau không tự sửa tổng tiền của đơn cũ.

hotel\_owner sở hữu đối tượng schema và chạy migration. hotel\_app chỉ có SELECT, INSERT, UPDATE, DELETE trên bảng cùng quyền sử dụng sequence. hotel\_monitor nhận pg\_monitor phục vụ exporter. Tách quyền giúp lỗi trong ứng dụng không mặc nhiên có quyền tạo role, tạo database hoặc thay đổi schema.

#### 4. WEBSITE VÀ NGHIỆP VỤ ĐẶT PHÒNG



Hình 3. Luồng tạo đặt chỗ và khóa bản ghi phòng.

Người dùng chưa đăng nhập được xem phòng; khi đặt chỗ phải đăng nhập. Form nhận tên khách, số điện thoại, ngày nhận, ngày trả và số khách. Server kiểm tra ngày không nằm trong quá khứ, ngày trả hợp lệ và số khách không vượt sức chứa. Tổng tiền được tính ở server, không sử dụng giá trị gửi từ trình duyệt.

Hai khoảng đặt phòng giao nhau khi check\_in cũ nhỏ hơn check\_out mới và check\_out cũ lớn hơn check\_in mới. Chỉ đơn confirmed tham gia kiểm tra. Khoảng ngày có biên phải mở, vì vậy một khách có thể nhận phòng đúng ngày khách trước trả phòng.

Trong transaction.atomic, select\_for\_update khóa bản ghi Room trước khi kiểm tra và lưu Booking. Các yêu cầu đặt cùng phòng phải chờ nhau trên PostgreSQL. Bộ test SQLite không chứng minh cơ chế khóa; scripts/concurrency.py được cung cấp để kiểm tra hai yêu cầu đồng thời trên hệ thống triển khai thật. Khách chỉ được xem và hủy đơn của mình; hủy dùng POST kèm CSRF.

## 5. DOCKER COMPOSE VÀ TRIỂN KHAI

Compose định nghĩa 12 service: db, migrate, web, nginx, pgadmin, prometheus, grafana, nginx-exporter, postgres-exporter, cadvisor, loki và promtail. Service migrate là tác vụ chạy một lần, kết thúc thành công trước khi web khởi động; các dịch vụ nền dùng restart unless-stopped.

| Bước | Thao tác                      | Kết quả cần xác nhận                  |
|------|-------------------------------|---------------------------------------|
| 1    | python3 scripts/setup.py      | .env mới với secret ngẫu nhiên        |
| 2    | docker compose config --quiet | Cấu hình hợp lệ, không in secret      |
| 3    | docker compose pull           | Tải được các image có tag             |
| 4    | docker compose up -d --build  | Build ứng dụng/proxy và khởi động     |
| 5    | docker compose ps -a          | migrate Exited 0; service nền Running |
| 6    | curl localhost:8080/health/   | status ok, database connected         |

### Khởi tạo dữ liệu

PostgreSQL chạy db/init.sh khi volume dữ liệu trống. Script tạo các role và default privileges. Django migrate tạo bảng; seed tạo sáu phòng mẫu và tài khoản admin nếu chưa tồn tại. Khi chạy lại seed không ghi đè tài khoản hoặc phòng đã có. collectstatic đưa static Django admin vào volume để Nginx phục vụ.

### Lưu trữ bền vững

Named volume lưu PostgreSQL, Grafana, pgAdmin, Prometheus, Loki và log nguồn. docker compose stop hoặc down thông thường không xóa volume. down -v xóa dữ liệu và chỉ được sử dụng khi chủ động muốn làm lại toàn bộ lab. Thay mật khẩu trong .env không tự đổi mật khẩu đã lưu trong PostgreSQL của volume cũ.

### Môi trường ưu tiên

Cấu hình cAdvisor sử dụng filesystem và thiết bị của Linux host. Vì vậy bài demo ưu tiên Ubuntu trong VMware với Docker Engine. Các cổng quản trị bind localhost; khi thao tác từ Windows sử dụng SSH tunnel như README để không phải mở trực tiếp pgAdmin, Grafana và Prometheus ra mạng.

## 6. NGINX REVERSE PROXY

Nginx lắng nghe cổng 8080 và proxy\_pass tới http://web:8000. Tên web được Docker DNS phân giải trong mạng dùng chung. Host header giữ thông tin host/port trình duyệt sử dụng để Django kiểm tra CSRF và ALLOWED\_HOSTS. X-Real-IP và X-Forwarded-Proto cung cấp thông tin nguồn và giao thức.

| Cấu hình                             | Vai trò                                 |
|--------------------------------------|-----------------------------------------|
| <code>/static/</code>                | Đọc static volume, không qua Django     |
| <code>/metrics bên ngoài</code>      | Trả 404; Prometheus truy cập web nội bộ |
| <code>8081/stub_status</code>        | Endpoint nội bộ cho Nginx exporter      |
| <code>server_tokens off</code>       | Hạn chế lộ phiên bản Nginx              |
| <code>client_max_body_size 1m</code> | Giới hạn kích thước request body        |

### Security headers

X-Content-Type-Options: nosniff yêu cầu trình duyệt không đoán kiểu nội dung. X-Frame-Options: DENY và frame-ancestors none ngăn nhúng trang trong iframe. Referrer-Policy: same-origin hạn chế truyền referrer sang website khác. Permissions-Policy vô hiệu camera, microphone và geolocation không sử dụng trong bài. CSP chỉ cho phép tài nguyên cùng nguồn; inline style được giữ vì giao diện dùng CSS nội tuyến.

Bản triển khai chọn phương án security headers theo đề bài, chưa cấp HTTPS. HTTP chỉ phù hợp VM lab; đăng nhập trên Internet cần TLS và cookie Secure. Không bật HSTS khi chưa có HTTPS vì sẽ tạo cách hiểu sai về bảo vệ đường truyền.

### Minh chứng

Thực hiện curl -I http://localhost:8080 và chụp các header trả về. Sau đó mở website qua cổng 8080 và ghi một đơn thành công. Chỉ mở ứng dụng trực tiếp ở cổng 8000 không chứng minh được reverse proxy; cổng này không được publish trong Compose.

## 7. GIÁM SÁT PROMETHEUS – GRAFANA

Prometheus dùng chu kỳ scrape 15 giây và lưu dữ liệu 15 ngày. Năm job bao gồm prometheus, nginx, postgres, containers và hotel. Mỗi job trỏ tới tên service trong mạng Docker; không dùng localhost để chỉ một container khác.

| Nguồn                      | Metric / cách dùng                                                              |
|----------------------------|---------------------------------------------------------------------------------|
| <b>cAdvisor</b>            | rate(container_cpu_usage_seconds_total[5m]); container_memory_working_set_bytes |
| <b>Nginx exporter</b>      | nginx_up;<br>rate(nginx_http_requests_total[5m]);<br>nginx_connections_active   |
| <b>PostgreSQL exporter</b> | pg_up; pg_stat_database_numbackends                                             |
| <b>Ứng dụng</b>            | hotel_rooms_total;<br>hotel_bookings_confirmed                                  |
| <b>Prometheus</b>          | up theo từng job                                                                |

Grafana provision sẵn datasource Prometheus và Loki, cùng dashboard Hotel · Operations. Dashboard có biểu đồ CPU/RAM, tốc độ request Nginx, số kết nối, khả dụng DB, kết nối DB, số đơn xác nhận và log Nginx. Sau khởi động cần tạo traffic và chờ ít nhất hai lần scrape; rate với khoảng 5 phút cần thêm thời gian để biểu đồ ổn định.

### Cách đọc đúng kết quả

up=1 chỉ chứng minh Prometheus lấy được trang metrics từ target. PostgreSQL exporter vẫn có thể trả metrics khi DB lỗi, vì vậy cần kiểm tra pg\_up=1. Tương tự phải kiểm tra nginx\_up=1. Biểu đồ container rỗng cần kiểm tra quyền đọc host và đường dẫn Docker; không kết luận hệ thống đạt chỉ vì dashboard đã xuất hiện.

### Cảnh báo

File alerts.yml có TargetDown khi target không được scrape trong một phút và PostgresDown khi pg\_up bằng 0. Các rule hiển thị trạng thái trong Prometheus; chưa cấu hình Alertmanager để gửi thông báo email. Khi demo có thể dừng một exporter, quan sát cảnh báo rồi khởi động lại.

## 8. LOG TẬP TRUNG LOKI - PROMTAIL

Nginx ghi access log dạng JSON với thời gian, method, path, status, bytes và duration. Django ghi sự kiện booking\_created và booking\_cancelled kèm mã đơn, mã phòng; không ghi tên hoặc số điện thoại khách vào log nghiệp vụ. Promtail đọc hai volume theo các label job=nginx và job=hotel rồi gửi HTTP push tới Loki.

| Mục đích           | LogQL                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------|
| Toàn bộ access log | <code>{job="nginx",filename="/var/log/nginx/access.log"}</code>                           |
| Lỗi HTTP           | <code>{job="nginx",filename="/var/log/nginx/access.log"}   json   status &gt;= 400</code> |
| Đơn mới            | <code>{job="hotel"}  = "booking_created"</code>                                           |
| Đơn hủy            | <code>{job="hotel"}  = "booking_cancelled"</code>                                         |

Để tạo dữ liệu minh chứng, truy cập trang chủ, mở một URL không tồn tại, tạo một đơn rồi hủy đơn. Trong Grafana Explore chọn datasource Loki và Last 15 minutes; thực hiện lần lượt ít nhất ba query. Ảnh cần thể hiện câu query, thời gian và dòng kết quả, không chỉ trang cấu hình datasource.

### Lưu trữ và giới hạn

Loki dùng TSDB schema v13, filesystem object store và compactor retention 720 giờ, tương đương 30 ngày. Khi báo cáo sau nhiều tuần, cần tạo dữ liệu demo mới thay vì kỳ vọng log cũ còn mãi. Retention của Loki không tự xóa file nguồn trong volume Nginx/ứng dụng; vận hành dài hạn cần cơ chế log rotation riêng.

Promtail đã hết vòng đời hỗ trợ từ 02/03/2026 theo Grafana. Thành phần vẫn được dùng để đáp ứng yêu cầu cụ thể của đề, trong môi trường lab. Với triển khai thực nên chuyển sang Grafana Alloy. Positions hiện lưu trong tmpfs, nên tạo lại Promtail có thể đọc lại log cũ; đây là giới hạn cần xử lý khi phát triển tiếp.

## 9. HARDENING HỆ THỐNG

| Biện pháp            | Cách áp dụng                                          | Kiểm tra                        |
|----------------------|-------------------------------------------------------|---------------------------------|
| <b>Non-root</b>      | Web UID 10001; Nginx UID 101; Promtail UID 10001      | docker compose exec web id      |
| <b>Tách mạng</b>     | data/observe internal; dịch vụ tham gia theo nhu cầu  | docker network inspect          |
| <b>Tách DB role</b>  | Owner, app DML, monitor                               | Truy vấn pg_roles và quyền bảng |
| <b>Giới hạn cổng</b> | Không publish DB/web/exporter; quản trị localhost     | docker compose ps / port        |
| <b>Giảm quyền</b>    | no-new-privileges, cap_drop ALL ở các service phù hợp | Đọc compose và inspect          |
| <b>Readonly</b>      | Web/Nginx filesystem chỉ đọc; tmpfs cho /tmp          | Kiểm tra ReadonlyRootfs         |
| <b>Secret</b>        | Sinh ngẫu nhiên, .env gitignored                      | git status; không in mật khẩu   |
| <b>Web security</b>  | CSRF, ORM, headers, session flags                     | Test CSRF và curl headers       |

Biện pháp bảo mật được áp dụng theo tính chất từng dịch vụ, không khẳng định mọi service có cùng quyền. PostgreSQL cần khởi tạo thư mục dữ liệu theo cơ chế của image. cAdvisor là ngoại lệ privileged để quan sát host và truy cập thiết bị; chỉ nên chạy trong VM thực hành tin cậy, không xuất cổng ra bên ngoài.

Mount /var/run ở chế độ read-only vẫn không biến Docker API thành read-only. Đây là một đánh đổi quyền quan sát của cAdvisor. Việc ghi rõ ngoại lệ giúp đánh giá thực chất hơn so với chỉ liệt kê một số từ khóa hardening mà bỏ qua quyền cao của agent giám sát.

Khi kiểm tra quyền DB, ứng dụng không được dùng tài khoản postgres. hotel\_app được phép thay đổi dữ liệu nghiệp vụ nhưng không có quyền DDL. Tách owner khỏi runtime giảm tác động của lỗi thực thi truy vấn. Thông tin .env phải được giữ riêng và không xuất hiện trong ảnh, báo cáo hoặc repository công khai.

## 10. KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ

Bộ kiểm thử tự động đã chạy thành công 15 test trên Django 5.2.7 với SQLite tách biệt. File docs/TEST\_RESULTS.txt lưu đầu ra thực tế. Các test kiểm tra hành vi nghiệp vụ và bảo vệ truy cập, không được dùng để kết luận toàn bộ hạ tầng Docker đã hoạt động.

| Nhóm                     | Ca kiểm thử                                | Kết quả local |
|--------------------------|--------------------------------------------|---------------|
| <b>Hiển thị</b>          | Trang chủ hiển thị phòng                   | Đạt           |
| <b>Đặt chỗ</b>           | Tính tiền server; trùng lịch; lịch liền kề | 3/3 đạt       |
| <b>Đầu vào</b>           | Ngày đảo ngược; ngày quá khứ; quá sức chứa | 3/3 đạt       |
| <b>Hủy</b>               | Hủy rồi đặt lại; bắt buộc POST             | 2/2 đạt       |
| <b>Quyền</b>             | Đơn người khác; login; nonstaff admin      | 3/3 đạt       |
| <b>Bảo vệ / vận hành</b> | CSRF; health/metrics; phòng inactive       | 3/3 đạt       |

### Kiểm thử tại máy triển khai

scripts/verify.py kiểm tra health, security headers, năm Prometheus target, pg\_up/nginx\_up, Grafana health và label log Nginx trong Loki. Script không thay thế kiểm tra trực quan biểu đồ và ba query LogQL. scripts/concurrency.py tạo tài khoản thử và gửi hai yêu cầu đặt cùng phòng đồng thời, yêu cầu một thành công và một bị từ chối.

### Kết quả còn phải bổ sung

Chưa có kết quả chạy Docker Engine, migration trên PostgreSQL, kết nối pgAdmin hoặc exporter trong môi trường bàn giao. Sinh viên cần chạy hai script trên VM, ghi ngày kiểm tra, kết quả và ảnh. Nếu có lỗi, giữ log lỗi, sửa cấu hình rồi chạy lại; không điền “Đạt” trước khi xác minh.

Tiêu chuẩn chấp nhận cuối cùng: đơn hiển thị đúng trong website và PostgreSQL; hai yêu cầu đồng thời không tạo hai đơn giao lịch; CPU/RAM/web/DB có dữ liệu; ít nhất ba LogQL trả dòng thực; dữ liệu tồn tại sau khi dừng và mở lại hệ thống.

## 11. GITHUB, VẬN HÀNH VÀ DEMO

Lịch sử Git đóng gói có ba mốc kỹ thuật: ứng dụng, database và Nginx; bổ sung hệ thống giám sát; bổ sung log tập trung, hardening và tài liệu. Các commit do Project Builder tạo để lưu trạng thái cấu hình, không phải minh chứng sinh viên đã thao tác triển khai. Sinh viên cần đẩy lên repository thuộc tài khoản theo MSSV và thêm kết quả chạy thật.

### Quy trình nộp

Tạo repo rỗng MSSV-Hotel-Booking trên GitHub, cấu hình tên/email Git của sinh viên, thêm remote origin rồi push main. README chứa hướng dẫn chạy và thông tin thành phần. Trước push kiểm tra git status, xác nhận .env và backup DB không bị theo dõi. Không đưa PAT vào URL remote hoặc vào commit.

### Vận hành

Sau khi khởi động lại VM, chạy docker compose up -d và kiểm tra health. docker compose logs --tail=100 tên\_service giúp chẩn đoán theo thành phần. Backup PostgreSQL bằng pg\_dump vào thư mục backups được gitignore. Trước thay đổi lớn cần sao lưu; quá trình restore phải chọn đúng DB đích, tránh ghi đè dữ liệu đang dùng.

| Thời lượng | Nội dung demo                                         |
|------------|-------------------------------------------------------|
| 1 phút     | Giới thiệu đề và sơ đồ kiến trúc                      |
| 3 phút     | Đăng ký, đặt phòng, chống trùng, hủy, bản ghi pgAdmin |
| 2 phút     | Prometheus targets và dashboard container/web/DB      |
| 2 phút     | Tạo log và chạy ba LogQL                              |
| 2 phút     | Hardening, GitHub, giải thích dữ liệu bền vững        |

Khi trình bày cần phân biệt metrics là số đo theo thời gian, log là bản ghi sự kiện; exporter chuyển số liệu của dịch vụ sang định dạng Prometheus; Grafana hiển thị chứ không thay PostgreSQL hoặc tự thu toàn bộ metrics. Reverse proxy là điểm vào của website, không phải cơ sở dữ liệu.

## 12. KẾT LUẬN VÀ MINH CHỨNG BỔ SUNG

Thiết kế Lotus Stay bao quát các nhóm yêu cầu: mã nguồn có cấu trúc, nghiệp vụ đặt phòng, PostgreSQL và pgAdmin, reverse proxy, metrics, log tập trung và các biện pháp hardening. Giá trị của bài thực hành nằm ở khả năng triển khai tái lập và giải thích được từng kết quả quan sát. Việc xác nhận đủ tiêu chí phải dựa trên lần chạy Docker thực tế, ảnh và repository của sinh viên.

### Danh sách ảnh cần chèn trước khi nộp

| Ảnh    | Nội dung                                    |
|--------|---------------------------------------------|
| A1     | GitHub: README và lịch sử commit            |
| A2     | docker compose ps -a                        |
| A3     | Website và đơn đặt phòng thành công         |
| A4     | pgAdmin: bản ghi booking_booking            |
| A5     | curl headers qua Nginx                      |
| A6     | Prometheus: 5 targets UP                    |
| A7     | Grafana: CPU/RAM, Nginx và PostgreSQL       |
| A8–A10 | Ba kết quả LogQL khác nhau                  |
| A11    | UID non-root, network isolation và quyền DB |

### Tài liệu tham khảo

Docker Compose: <https://docs.docker.com/compose/>

Django transactions: <https://docs.djangoproject.com/en/5.2/topics/db/transactions/>

PostgreSQL exporter: [https://github.com/prometheus-community/postgres\\_exporter](https://github.com/prometheus-community/postgres_exporter)

Nginx exporter: <https://github.com/nginx/nginx-prometheus-exporter>

cAdvisor: <https://github.com/google/cadvisor>

Loki retention: <https://grafana.com/docs/loki/latest/operations/storage/retention/>

Promtail lifecycle: <https://grafana.com/docs/loki/latest/send-data/promtail/>